

Informe Técnico Integral

Fundamentos de Desarrollo de Software Moderno

Índice General

- 1. Introducción a REST APIs
- 2. Terminal y Línea de Comandos
- 3. Estructura de Proyectos Frontend y Backend
- 4. Next.js App Router
- 5. Arquitecturas de Software
- 6. Lógica de Programación Avanzada
- 7. Git: Branching y Estrategias de Merge

1. Introducción a REST APIs

1.1 ¿Qué es REST?

REST (Representational State Transfer) es un estilo arquitectónico para diseñar servicios web que se basa en el protocolo HTTP. No es un protocolo en sí mismo, sino un conjunto de principios que guían cómo deben comunicarse los sistemas.

1.2 Principios Fundamentales

Principio	Descripción
Cliente-Servidor	Separación clara entre la interfaz de usuario (cliente) y la lógica/almacenamiento (servidor)
Stateless	Cada petición contiene toda la información necesaria; el servidor no almacena estado del cliente
Cacheable	Las respuestas pueden almacenarse en caché para mejorar el rendimiento
Interfaz Uniforme	Interacción consistente con los recursos mediante métodos HTTP estándar
Sistema por Capas	El cliente no sabe si está conectado directamente al servidor final o a un intermediario
Código bajo Demanda (Opcional)	El servidor puede enviar código ejecutable al cliente

1.3 Recursos y URIs

En REST, todo es un recurso. Cada recurso se identifica mediante una URI única:

```
https://api.example.com/users/123
```

1.4 Métodos HTTP (CRUD)

Método	Acción	Descripción	Idempotente
GET	Leer	Recuperar un recurso	Sí
POST	Crear	Crear un nuevo recurso	No
PUT	Actualizar	Actualizar recurso completo	Sí
PATCH	Actualizar	Actualizar parte del recurso	No
DELETE	Eliminar	Eliminar un recurso	Sí

1.5 Ejemplos de Endpoints REST

Operación	Método	Endpoint	Ejemplo
Obtener todos los usuarios	GET	/users	GET /users
Obtener usuario por ID	GET	/users/{id}	GET /users/123
Crear usuario	POST	/users	POST /users
Actualizar usuario	PUT	/users/{id}	PUT /users/123
Actualizar parcialmente	PATCH	/users/{id}	PATCH /users/123
Eliminar usuario	DELETE	/users/{id}	DELETE /users/123

1.6 Códigos de Estado HTTP Comunes

Código	Descripción
200	OK
201	Creado
400	Solicitud Incorrecta
401	No Autorizado
403	Prohibido
404	No Encontrado

1.7 Buenas Prácticas para REST APIs

- Usar sustantivos para recursos, no verbos
- Usar plurales para colecciones (ej. /users)
- Utilizar los métodos HTTP adecuados
- Versionar la API (ej. /v1/users)
- Usar HTTPS para comunicación segura
- Proporcionar mensajes de error descriptivos
- Incluir documentación clara (OpenAPI/Swagger)

1.8 Herramientas para Probar APIs REST

- Postman
- Insomnia
- cURL
- Swagger/OpenAPI
- VS Code REST Client

2. Terminal y Línea de Comandos

2.1 ¿Qué es la Terminal?

La terminal (o consola) es un programa que proporciona una interfaz de texto para interactuar con el sistema operativo. Permite ejecutar comandos para navegar archivos, gestionar configuraciones, instalar software y automatizar tareas.

2.2 Componentes de la Terminal

```
[usuario@máquina: ~]$ comando -opción argumento
```

- **Terminal Emulator:** La aplicación que usas (Windows Terminal, iTerm2, GNOME Terminal)
- **Shell:** El programa que interpreta tus comandos (bash, zsh, fish, PowerShell)

2.3 Shells Comunes

Shell	Características
bash	Predeterminado en Linux y macOS
zsh	Predeterminado en macOS moderno
fish	Interfaz amigable y autocompletado avanzado
PowerShell	Potente shell para Windows

2.4 Comandos Básicos Esenciales

Comando	Descripción
ls	Listar archivos y directorios
cd <dir>	Cambiar directorio actual
mkdir <dir>	Crear un nuevo directorio
rm <file>	Eliminar un archivo
rm -r <dir>	Eliminar directorio y contenido
cp <src> <dest>	Copiar archivos/directorios
mv <src> <dest>	Mover o renombrar archivos
cat <file>	Mostrar contenido de un archivo
touch <file>	Crear archivo vacío
clear	Limpiar la pantalla
man <cmd>	Mostrar manual del comando
exit	Cerrar la terminal

2.5 Consejos para Usar la Terminal

- Usar Tabulador para autocompletar
- Usar ↑ y ↓ para navegar por el historial
- Aprender un comando a la vez
- Ser cuidadoso con comandos destructivos (como `rm -rf /`)

2.6 ¿Por Qué Aprender la Terminal?

- Más rápido que hacer clic
- Mayor control sobre el sistema
- Esencial para desarrollo
- Funciona en servidores locales y remotos
- Permite automatización y productividad

3. Estructura de Proyectos Frontend y Backend

3.1 Frontend (React/Angular/Vue)

Responsabilidades: Desarrollo de UI, Diseño Responsive, Integración de APIs, Gestión de Estado, Validación de Formularios, Enrutamiento, Autenticación UI.

Estructura Típica:

```
src/
├─ components/      # Componentes reutilizables
├─ pages/           # Páginas/rutas
├─ hooks/           # Custom hooks (React)
├─ context/         # Context API (AuthContext, etc.)
├─ store/           # Redux/State Management
├─ routes/          # Definición de rutas
├─ utils/           # Funciones auxiliares
├─ assets/          # Imágenes, fuentes
├─ themes/          # Configuración de temas
├─ validators/      # Validaciones del lado cliente
├─ @types/          # Definiciones de tipos TS
├─ app.tsx          # Componente principal
└─ main.tsx         # Punto de entrada
```

3.2 Backend (Node.js/Express)

Responsabilidades: Desarrollo de REST APIs, Gestión de bases de datos, Autenticación y Autorización, Lógica de negocio, Seguridad, Subida de archivos, Servicios de email, Integración de pasarelas de pago.

Estructura Típica:

```
src/
├─ api/             # Rutas de la API
├─ controllers/     # Controladores (lógica de peticiones)
├─ services/        # Servicios (lógica de negocio)
├─ models/          # Modelos de datos
├─ middlewares/     # Middlewares (auth, errores)
├─ helpers/         # Funciones de ayuda
├─ validators/      # Validaciones de datos
├─ database/        # Configuración DB, migraciones
├─ utils/           # Utilidades (jwt, logger)
├─ routes/          # Definición de rutas
├─ config/          # Configuraciones
├─ prisma/          # (si se usa Prisma)
└─ server.ts        # Servidor principal
```

3.3 Full Stack Project Structure

```
fullstack-project/  
├── client/           # Frontend (React/Angular/Vue)  
├── server/           # Backend (Node.js/Java/Spring/.NET)  
├── docs/             # Documentación  
├── docker/           # Configuración Docker  
├── .env              # Variables de entorno  
├── .gitignore        # Archivos ignorados por Git  
├── package.json      # Dependencias  
├── biome.json        # Configuración linter  
├── README.md         # Documentación del proyecto  
└── tailwind.config.ts # (si se usa Tailwind)
```

3.4 Flujo de Aplicación

```
Frontend (React/Angular)  
  ↓  
  REST API / GraphQL  
  ↓  
Backend (Node.js/Java/Spring/.NET)  
  ↓  
Database (MySQL/PostgreSQL/MongoDB)
```

3.5 Separación de Responsabilidades

- **Controllers:** Manejan las peticiones HTTP y respuestas
- **Services:** Contienen la lógica de negocio
- **Models:** Representan la estructura de datos

¿Por qué separar Controllers, Services y Models?

Esta separación hace que la aplicación sea escalable, mantenible y más fácil de probar. Permite cambios en una capa sin afectar a las otras.

4. Next.js App Router

4.1 Estructura Simplificada

```
my-next-app/
├── public/           # Archivos estáticos
│   ├── images/
│   ├── fonts/
│   └── logo.png
├── src/
│   ├── app/         # App Router Core
│   │   ├── layout.tsx # Layout raíz compartido
│   │   ├── page.tsx   # Página principal (/)
│   │   ├── loading.tsx # UI de carga
│   │   ├── error.tsx  # Manejo de errores
│   │   ├── api/       # Rutas API
│   │   │   └── hello/
│   │   │       └── route.ts
│   │   └── blog/      # Ruta: /blog
│   │       ├── layout.tsx # Layout anidado
│   │       ├── page.tsx  # Índice del blog
│   │       └── [slug]/   # Ruta dinámica: /blog/post-1
│   │           └── page.tsx
│   ├── components/   # Componentes reutilizables
│   │   ├── Header.tsx
│   │   ├── Footer.tsx
│   │   └── Button.tsx
│   └── ...
├── package.json
├── next.config.js
├── .env.local
├── tsconfig.json
└── .gitignore
```

4.2 Convención de Colores

Color	Significado
 Azul	Directorios
 Verde	Páginas/Layouts/Componentes UI
 Amarillo	Archivos de configuración
 Púrpura	Rutas API/Lógica de servidor

4.3 Características Clave del App Router

- Layouts anidados: UI compartida en rutas específicas
- Server Components por defecto
- Carga optimizada con `loading.tsx`

- Manejo de errores con `error.tsx`
- Rutas API dentro del directorio `app/api/`
- Rutas dinámicas con `[slug]`

5. Arquitecturas de Software

5.1 12 Estilos de Arquitectura

Arquitectura	Descripción	Caso de Uso
Monolítica	Un solo bloque con toda la funcionalidad	Proyectos pequeños, equipos reducidos
Capas (N-Tier)	Presentación → Lógica → Datos → DB	Aplicaciones empresariales tradicionales
Cliente-Servidor	Clientes solicitan recursos a servidores	Aplicaciones web y móviles
Microservicios	Múltiples contenedores independientes	Sistemas grandes, escalables, equipos distribuidos
SOA	Servicios conectados a un bus central (ESB)	Integración empresarial
Event-Driven	Flujo asíncrono de eventos entre componentes	Sistemas reactivos, procesamiento en tiempo real
Serverless	Funciones ejecutadas bajo demanda en la nube	Aplicaciones de bajo costo y escalables
Microkernel	Núcleo mínimo + plugins extensibles	Aplicaciones extensibles (IDE, ERP)
Hexagonal	Núcleo aislado con adaptadores	Aplicaciones con alta complejidad de dominio
Peer-to-Peer	Nodos interconectados compartiendo recursos	Sistemas descentralizados, blockchain
Basada en Componentes	Bloques modulares reutilizables	Sistemas con alta reusabilidad
Broker/Pipeline	Flujo secuencial con intermediarios	Procesamiento de datos, ETL

5.2 Comparativa Rápida

Arquitectura	Escalabilidad	Complejidad	Mantenimiento
Monolítica	Baja	Baja	Simple al inicio, compleja al crecer
Microservicios	Alta	Alta	Desafiante pero flexible
Serverless	Muy Alta	Media	Menos gestión de infraestructura

Hexagonal	Alta	Alta	Excelente para dominios complejos
-----------	------	------	-----------------------------------

6. Lógica de Programación Avanzada

6.1 Optimización Algorítmica

De "Hacer Código" a "Diseñar Arquitecturas"

Concepto	Ineficiente	Optimizado
Complejidad	$O(n^2)$	$O(1) / O(\log n)$
Ramas/Ciclos	Múltiples anidados	Minimizados
Pruebas	Solo casos base	Pruebas de borde avanzadas

6.2 Patrones de Diseño y SOLID

Patrones Clave:

- **Observer:** Notificación de cambios a múltiples objetos
- **Strategy:** Algoritmos intercambiables en tiempo de ejecución
- **Dependency Injection:** Inyección de dependencias para desacoplamiento

Principios SOLID:

- Single Responsibility
- Open/Closed
- Liskov Substitution
- Interface Segregation
- Dependency Inversion

Beneficio: Acoplamiento débil y cohesión fuerte

6.3 Concurrencia y Gestión de Memoria

Desafíos:

- **Carreras de Datos:** Condiciones de competencia entre hilos
- **Deadlocks:** Bloqueos mutuos entre procesos
- **Gestión de Hilos:** Sincronización y pooling
- **Recolección de Basura:** Sintonización para rendimiento

6.4 Arquitectura Escalable

Componentes Clave:

- **Service Discovery:** Registro y descubrimiento de servicios
- **Message Queues:** Comunicación asíncrona entre servicios
- **Load Balancing:** Distribución de carga avanzada

- **Caché de Capas:** Almacenamiento en múltiples niveles
- **Sistemas Distribuidos:** Coordinación entre nodos

7. Git: Branching y Estrategias de Merge

7.1 ¿Por Qué Usar Branches?

- Desarrollar funcionalidades en aislamiento
- Experimentar sin afectar el código principal
- Facilitar la colaboración en equipo
- Mantener la rama principal estable
- Facilitar code review e integración CI/CD

7.2 Tipos de Branches

Branch	Propósito
main / master	Código estable y listo para producción
feature	Nueva funcionalidad o tarea
release	Preparación para lanzamiento a producción
hotfix	Correcciones urgentes en producción

7.3 Estrategias de Merge

Fast-Forward Merge

Antes: A ← B ← C (feature) ← main
 Después: A ← B ← C (feature/main)

- Pros: Historial limpio
- Contras: Pierde contexto de la rama

Merge Commit (No-FF)

Antes: A ← B ← C (feature) ← main
 Después: A ← B ← C ← M (merge commit) ← main

- Pros: Mantiene historial completo
- Contras: Más commits

Rebase

Antes: A ← B ← C (feature) ← main
 Después: A ← B ← C' (feature) ← main

- Pros: Historial lineal y limpio
- Contrás: Reescribe historia (riesgoso en branches compartidas)

7.4 Comparativa Merge vs Rebase

Aspecto	Merge	Rebase
Historial	Mantiene todo (incluye commits de merge)	Historial lineal
Commits	No se reescriben	Se reescriben
Mejor para	Branches compartidas	Branches locales
Seguridad	Seguro	Riesgoso en shared branches

7.5 Flujos de Trabajo Comunes

Git Flow:

- Usa: main, develop, feature, release, hotfix
- Mejor para: Proyectos grandes con lanzamientos programados

GitHub Flow:

- Feature branches → Pull Request → Review → Merge
- Mejor para: Entrega continua

Trunk-Based Development:

- Branches de corta duración, merges frecuentes a main
- Mejor para: Equipos de alta velocidad

7.6 Resolución de Conflictos

1. Git detecta conflicto durante merge/rebase
2. Abrir archivos conflictivos (marcados con <<<<<<<)
3. Editar y resolver manualmente
4. Añadir archivos resueltos: `git add .`
5. Continuar merge: `git commit o git rebase --continue`
6. Siempre probar el código después de resolver conflictos

7.7 Comandos Útiles de Git

```
git branch           # Listar branches
git branch <nombre>  # Crear branch
git checkout <branch> # Cambiar de branch
git checkout -b <branch> # Crear y cambiar
git merge <branch>    # Fusionar branch
git rebase <branch>   # Rebase branch
git push origin <branch> # Subir branch
git branch -d <branch> # Eliminar branch local
```

7.8 Buenas Prácticas

- Mantener main siempre desplegable
- Usar feature branches para todo nuevo trabajo
- Escribir mensajes de commit claros
- Hacer pull antes de empezar a trabajar
- Revisar código mediante Pull Requests
- Eliminar branches después de fusionar

Glosario de Términos Clave

Término	Definición
API	Application Programming Interface
REST	Representational State Transfer
URI	Uniform Resource Identifier
CRUD	Create, Read, Update, Delete
JSON	JavaScript Object Notation
CI/CD	Continuous Integration / Continuous Deployment
Shell	Intérprete de comandos
ESB	Enterprise Service Bus
SOLID	Principios de diseño de software
Cache	Almacenamiento temporal de datos
Idempotente	Operación que puede repetirse sin cambiar el resultado

Referencias y Recursos Adicionales

- Documentación REST: MDN Web Docs
- Git: Pro Git Book
- Next.js: Documentación Oficial
- Patrones de Diseño: "Design Patterns" - Gang of Four
- Arquitectura de Software: "Software Architecture in Practice"
- Herramientas: Postman, VS Code, Docker